Mini Static Findings Scanner

This is a mini command-line tool that scans a folder of source files and reports likely security and code-quality problems. It stays simple and explainable, everything is plain regex plus a few deterministic heuristics, no machine learning, so every finding traces back to a specific rule you can read. It runs fully offline by default and the only network call is the optional dependency check (`--online`), which looks up known CVEs from OSV.dev and sends nothing but package names and versions.

1. Highlights

1.1 **15 rules** covering secrets, dangerous calls, weak crypto, injection, CORS, debug config, sensitive files and more, plus an opt-in dependency CVE check against OSV.dev.

1.2 **A confidence score on every finding**, tracked separately from severity, so results can be triaged instead of just listed.

1.3 **A false-positive validator layer** that drops the usual noise (placeholders, env-var reads, example URLs, `# nosec` lines, and risky calls that only appear in comments or docstrings).

1.4 **Five outputs**: console, JSON, Markdown, SARIF (for GitHub code scanning), and a self-contained interactive HTML report.

1.5 **Offline by default**: the only network call is the dependency check, it is opt-in, and it never sends your code.

1.6 **Process-parallel scanning** that kicks in automatically past about 2,500 files, a threshold set from measurement, so large trees scan across all cores while small ones stay sequential and fast.

2. Quick start

You need Python 3.8 or newer. The only runtime dependency is `packaging`. Run the setup script once: it creates a virtual environment in `.venv`, installs the pinned dependency, and installs the `scanner` command.

2.1 Windows (PowerShell):

```
.\setup.ps1
.\.venv\Scripts\Activate.ps1
```

2.2 macOS / Linux:

```
./setup.sh
source .venv/bin/activate
```

If PowerShell blocks the script, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` once first. If you would rather not install anything, run it straight from the repo with `python main.py`

`<folder>` instead of the `scanner` command.

2.3 Then scan the included sample project:

```
scanner ./sample-project
```

It prints a table and writes `findings.json` and `findings-report.md` next to wherever you run it:

```
 #  SEVERITY  LOCATION            SHORT EXPLANATION
--  --------  ------------------  -----------------
 1  HIGH      src/config.js:1     Possible hardcoded secret or credential found
 2  MED       src/server.js:2     Broad CORS policy allows requests from any
origin
 ...
```

3. How it works

3.1 It walks the folder, skipping the usual noise (`.git`, `node_modules`, `dist`, `build`, virtual envs, caches).

3.2 Each rule is a regex plus optional context keywords and a list of validators. The regex finds candidates; the validators decide whether they are real.

3.3 The validators are what keep the noise down. They drop obvious false positives and adjust a confidence score from 0 to 1. Severity comes from the rule ("how bad if it's real"); confidence is separate ("how sure it is real").

3.4 With `--online`, it parses `requirements.txt` / `package.json` and asks the OSV.dev database whether any pinned version has a known CVE, sending only the package name and version.

3.5 Findings are de-duplicated, ranked by severity then confidence, printed, and written to whichever report formats you ask for.

4. Modes and flags

The default run is offline. If you name no format it writes JSON and Markdown. If you name one or more formats it writes only those.

4.1 `--online`: enable dependency CVE lookups via OSV.dev. Off by default; the only flag that uses the network.

4.2 `-v`, `--verbose`: show the rule name, CWE, confidence and fix under each finding.

4.3 `--min-confidence N`: drop findings below confidence N (0.0 to 1.0).

4.4 `-j N`, `--jobs N`: how many worker processes to scan with. Defaults to the CPU count.

4.5 `--config FILE`: load a JSON config (also auto-loads `scanner.config.json` if present).

4.6 `--enable-rule "Name"`: run only this rule. Repeatable.

4.7 `--disable-rule "Name"`: turn a rule off. Repeatable.

4.8 `--list-rules`: print every rule name and exit.

4.9 `--json [PATH]` writes a JSON report. It uses PATH if you give one, otherwise `findings.json`.

4.10 `--md [PATH]` writes a Markdown report. It uses PATH if you give one, otherwise `findings-report.md`.

4.11 `--sarif [PATH]` writes a SARIF report. It uses PATH if you give one, otherwise `findings.sarif`.

4.12 `--html [PATH]` writes an HTML report, opens it, and prints a link. It uses PATH if you give one, otherwise `findings.html`.

Examples:

```
scanner ./sample-project --online                  # include dependency CVEs
scanner ./sample-project --min-confidence 0.6      # only the confident findings
scanner ./sample-project -v                         # full detail in the console
scanner ./sample-project --html                     # open the review UI
scanner ./sample-project --disable-rule "Insecure HTTP URL"
scanner --list-rules
```

5. Output formats

The scanner writes only the formats you name. With no format flag it writes JSON and Markdown.

5.1 **Console**. A numbered table with severity, the file and line, and the short explanation. Add `-v` for the rule name, CWE, confidence, and fix.

5.2 **JSON**. Default file `findings.json`. Every field for each finding plus a summary count.

5.3 **Markdown**. Default file `findings-report.md`. Grouped by severity, with each finding shown with its location, CWE, confidence, and fix. The name keeps it from clashing with `FINDINGS.md` on case-insensitive filesystems.

5.4 **SARIF**. Pass `--sarif`. Standard 2.1.0 output that GitHub code scanning and the VS Code SARIF viewer understand.

5.5 **HTML**. Pass `--html`. A single self-contained page you can filter and search, with remediation shown inline. It opens in your browser, and the file link is printed to the console in case it does not. No server runs, and nothing leaves your machine.

The `examples/` folder has a checked-in report in every format from scanning `sample-project/` with `--online`, so you can see each one without running anything. The live `findings.*` files the tool writes are gitignored, since on a real codebase a report can quote real secrets.

6. Rules

Fifteen rules cover secrets, dangerous calls, insecure deserialization, weak crypto, SQL injection, path traversal, TLS verification, CORS, debug config, insecure URLs, sensitive files and suspicious comments. Run `scanner --list-rules` for the full list, or see `FINDINGS.md` for the breakdown. Those fifteen are the offline rules. With `-`

`-online`, the dependency check adds its own findings for known-vulnerable package versions and unpinned dependencies, which sit outside the rule list.

7. Config file

Drop a `scanner.config.json` next to where you run the tool (it is picked up automatically), or pass one with `--config`:

```
{
  "min_confidence": 0.0,
  "enabled_rules": [],
  "disabled_rules": ["Insecure HTTP URL"]
}
```

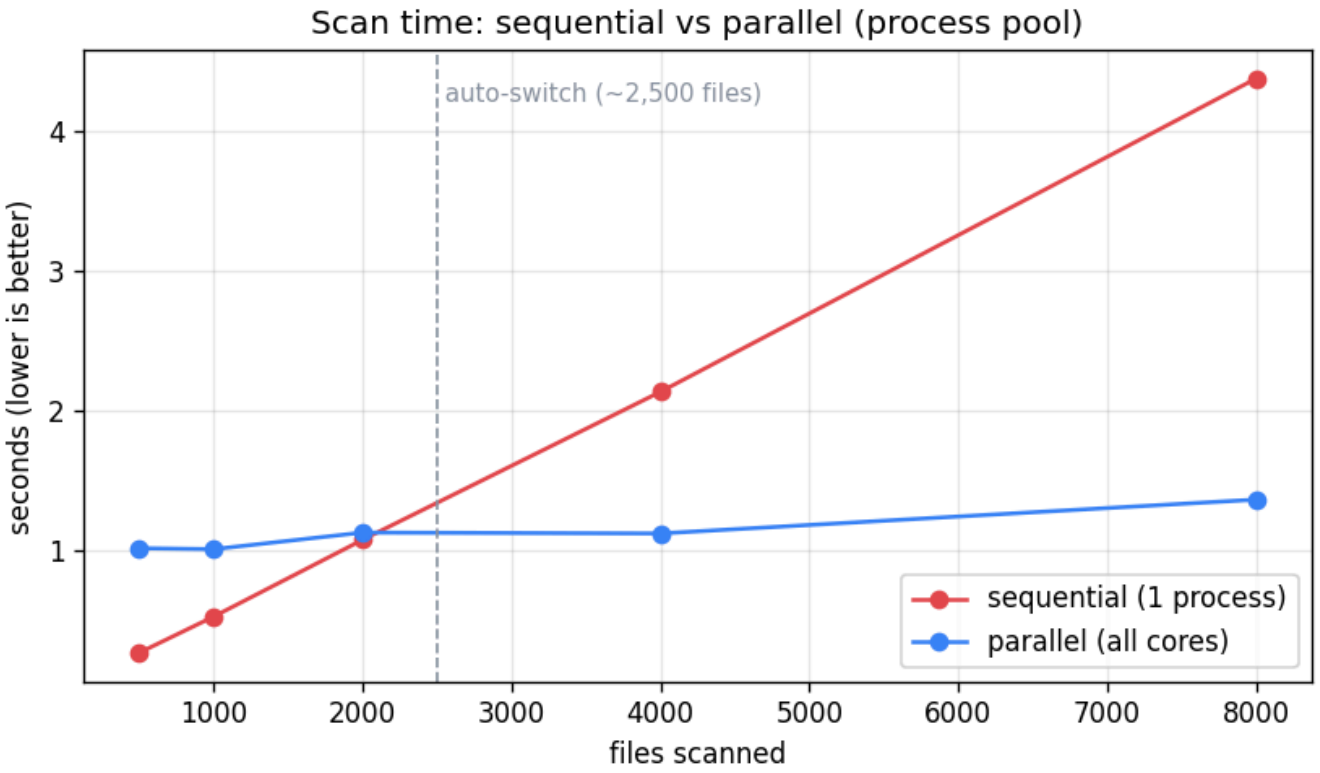
`enabled_rules` is an allowlist (if it is non-empty, only those rules run). `disabled_rules` turns rules off. The `--enable-rule` / `--disable-rule` flags merge with whatever is in the file.

8. Performance

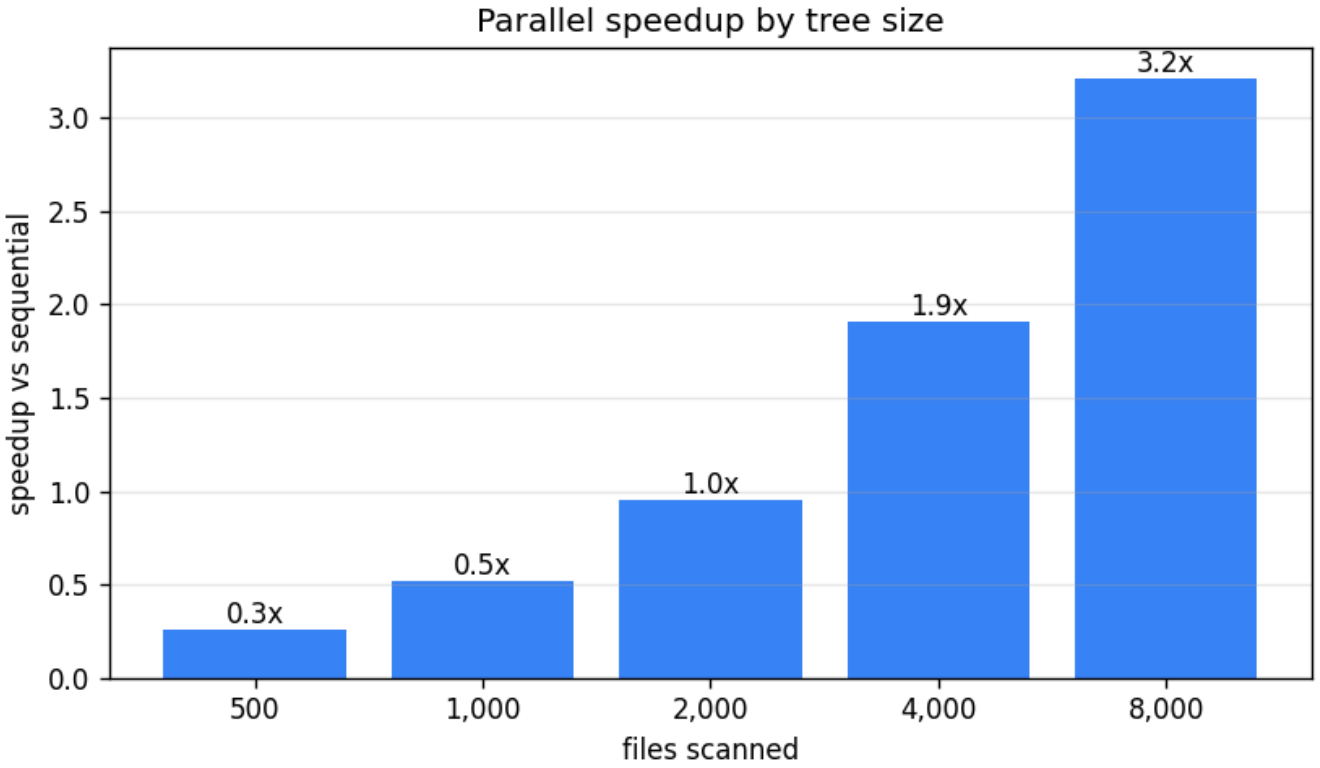
All numbers below were measured on this machine, Python 3.11 with 20 logical cores, using the committed benchmark script. You can regenerate them any time with `python benchmarks/benchmark.py`. It needs matplotlib, which the scanner itself does not, and the charts are committed so you do not have to run it.

8.1 Single-core throughput is around 1,900 files per second on the small synthetic files the benchmark uses. Larger real-world files scan slower per file, since the work is done line by line.

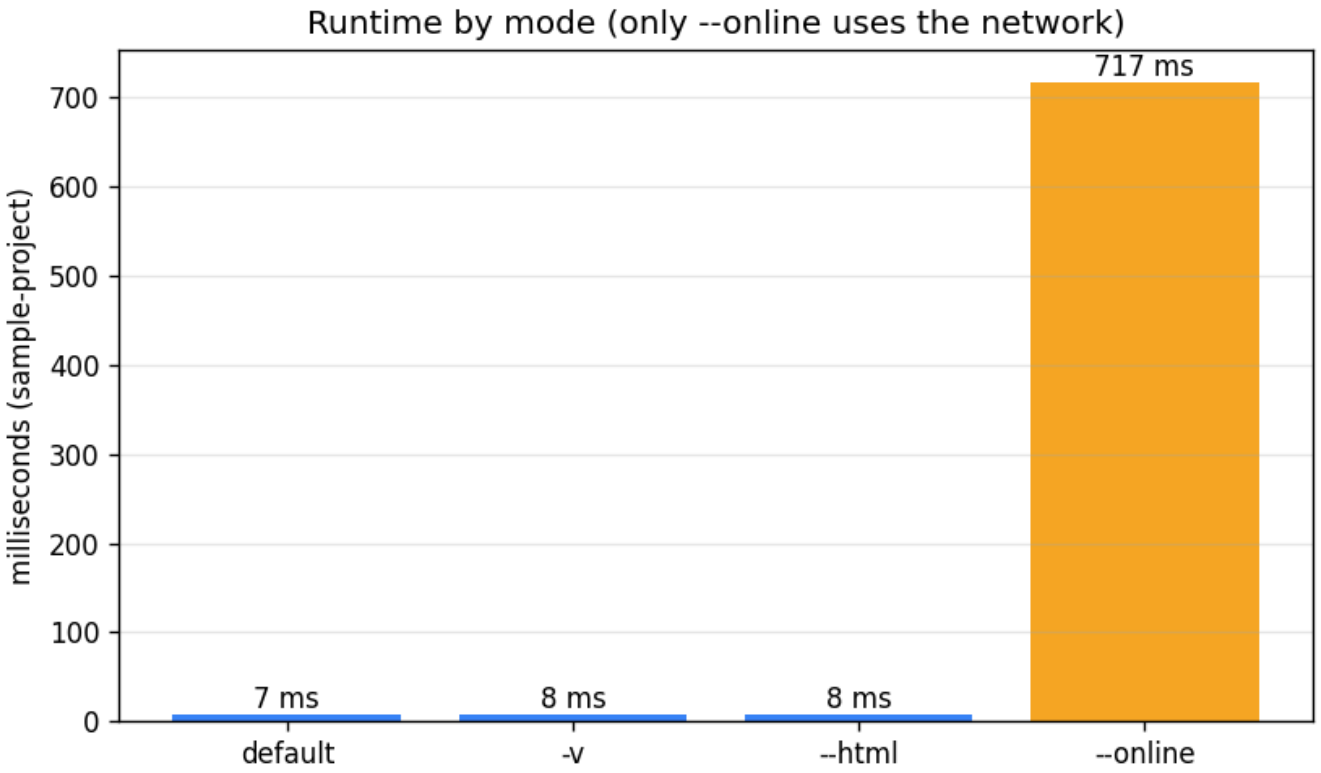
8.2 **Parallelism that switches on automatically.** File scanning is CPU-bound regex work, which does not speed up with threads because of Python's GIL, so the scanner uses a process pool instead. Spawning processes costs about a second on Windows, so for small and medium trees a plain sequential pass is actually faster. The scanner handles this for you. It stays sequential until the tree crosses a size threshold, the `PARALLEL_THRESHOLD` constant set to 2,500 files in `engine.py`, and only then spreads the work across all cores. Sequential and parallel break even at roughly 2,000 files. The 2,500 cutoff sits a little above that, so the tool only switches when the parallel win is clear rather than marginal. Past that point parallel stays flat while sequential keeps climbing.



The same data as a speedup factor makes the rule obvious. Below the threshold the process pool is a net loss, which is why the tool does not use it there. Above it the win grows with the tree, reaching about 3.2x at 8,000 files.



8.3 **Cost by mode.** Every mode runs the sample project in a few milliseconds, except for `--online`. The dependency CVE lookups are network round-trips to OSV, which dominate everything else and add several hundred milliseconds, sometimes over a second on a slow link, depending on latency. That is why the network is opt-in rather than on by default. The `-v` and `--html` modes add no meaningful overhead.



9. Tests

```
python -m unittest discover -s tests
```

Over 70 tests cover detection, false positive handling, dependency logic, the report formats, and the scanner's own hardening. Each area has its own file.

Test file	What it checks
`test_rules.py`	Each detection rule fires on a known bad sample.
`test_validators.py`	The false positive validators drop the noise and keep the real hits.
`test_sca.py`	Manifest parsing, OSV severity mapping, and dependency scanning with the network mocked.
`test_config.py`	Turning rules on and off through the config file and the CLI flags.
`test_languages.py`	Language detection from file extensions.
`test_sarif.py`	SARIF output has the right envelope, results, severity levels, and deduped rules.
`test_html.py`	The HTML report is self-contained and its embedded data is valid JSON.
`test_parallel.py`	The parallel and sequential scans return the same findings.
`test_output.py`	The format flags write only what you name, and with no flag it writes JSON and Markdown.
`test_security.py`	The scanner's own hardening, the per-line ReDoS cap and symlink containment.

Test file	What it checks
`test_edge_cases.py`	Empty, binary, oversized, and unicode files, broken manifests, online mode with no internet, permission errors, broken symlinks, and bad CLI arguments.

The scanner stays stable in every edge case above. The symlink tests are skipped on Windows unless Developer Mode is on, since creating a symlink needs elevated privileges.

10. Beyond the brief

These go beyond what the brief asked for. I added them to make the tool more practical and to show how I would approach the problem in a production setting:

10.1 **Dependency (SCA) scanning** against the OSV.dev database, so the tool catches known-vulnerable package versions, not just risky code.

10.2 **A confidence score** on every finding, kept separate from severity, so results can be triaged rather than dumped in a flat list.

10.3 **Process-parallel scanning with a threshold I measured rather than guessed.** I benchmarked sequential vs threads vs processes, found the GIL caps threaded regex, and set the auto-switch point from the actual crossover (the Performance charts above).

10.4 **A reproducible benchmark** (`benchmarks/benchmark.py`) that generates those charts, so the performance claims are backed by measurements.

10.5 **Hardening for scanning untrusted code**: a per-line length cap so a crafted line can't stall a regex (ReDoS), and symlink handling that won't follow links outside the target folder.

10.6 **A one-command setup** (`setup.ps1` / `setup.sh`) and a pinned dependency for reproducible installs.

11. Project layout

```
main.py                 thin entry point (python main.py ...)
scanner/
  cli.py                argument parsing and the run flow
  engine.py             file walking, scanning, parallelism, dedup, ranking
  rules.py              the rule definitions
  validators.py         false-positive validators and confidence scoring
  entropy.py            Shannon entropy helper
  languages.py          file-extension language detection
  sca.py                dependency parsing and OSV lookups
  reporter.py           console, JSON, Markdown, SARIF and HTML output
  schema.py             the Rule and Finding data structures
sample-project/         a small project with planted issues to scan
examples/               generated example reports (json, md, sarif, html)
benchmarks/             benchmark script and the charts in this README
tests/                  unit tests
setup.sh / setup.ps1    one-command bootstrap
```

See `FINDINGS.md` for the writeup on the rules, the false positives and negatives, and how I would prioritize findings in practice.

12. Acknowledgement

Thank you for the assignment. I really enjoyed working on this project and found it to be a useful exercise in balancing detection coverage, false-positive reduction, performance, and developer usability while staying aligned with the assignment scope. While there are several possible extensions, I focused on delivering a practical and well-tested implementation within the brief.

Sincerely,

Prahar Shah